

네트워크 보안 - PCAP Programming 과제

[코드 전체]

```
#include <stdlib.h>
#include <stdio.h>
#include <pcap.h>
#include <netinet/ether.h>
#include <arpa/inet.h>

/* Ethernet header */
struct ethheader {
    u_char ether_dhost[6]; /* destination host address */
    u_char ether_shost[6]; /* source host address */
    u_short ether_type;    /* IP? ARP? RARP? etc */
};

/* IP Header */
struct ipheader {
    unsigned char    iph_ihl:4, //IP header length
                   iph_ver:4; //IP version
    unsigned char    iph_tos; //Type of service
    unsigned short int iph_len; //IP Packet length (data + header)
    unsigned short int iph_ident; //Identification
    unsigned short int iph_flag:3, //Fragmentation flags
                   iph_offset:13; //Flags offset
    unsigned char    iph_ttl; //Time to Live
    unsigned char    iph_protocol; //Protocol type
    unsigned short int iph_chksum; //IP datagram checksum
    struct in_addr    iph_sourceip; //Source IP address
    struct in_addr    iph_destip; //Destination IP address
};
```

```

/* ICMP Header */
struct icmpheader {
    unsigned char icmp_type; // ICMP message type
    unsigned char icmp_code; // Error code
    unsigned short int icmp_chksm; //Checksum for ICMP Header and data
    unsigned short int icmp_id; //Used for identifying request
    unsigned short int icmp_seq; //Sequence number
};

/* TCP Header */
struct tcpheader {
    u_short tcp_sport;      /* source port */
    u_short tcp_dport;      /* destination port */
    u_int  tcp_seq;         /* sequence number */
    u_int  tcp_ack;         /* acknowledgement number */
    u_char tcp_offx2;       /* data offset, rsvd */
#define TH_OFF(th) (((th)→tcp_offx2 & 0xf0) >> 4)
    u_char tcp_flags;
#define TH_FIN 0x01
#define TH_SYN 0x02
#define TH_RST 0x04
#define TH_PUSH 0x08
#define TH_ACK 0x10
#define TH_URG 0x20
#define TH_ECE 0x40
#define TH_CWR 0x80
#define TH_FLAGS (TH_FIN|TH_SYN|TH_RST|TH_ACK|TH_URG|TH_ECE|TH_CWR)
    u_short tcp_win;        /* window */
    u_short tcp_sum;        /* checksum */
    u_short tcp_urp;        /* urgent pointer */
};

/* Psuedo TCP header */
struct pseudo_tcp
{
    unsigned saddr, daddr;
    unsigned char mbz;
};

```

```

    unsigned char ptcl;
    unsigned short tcpl;
    struct tcpheader tcp;
    char payload[1500];
};

void got_packet(u_char *args, const struct pcap_pkthdr *header,
               const u_char *packet)
{
    struct ethheader *eth = (struct ethheader *)packet;

    printf(" ether source : %s\n", ether_ntoa((struct ether_addr *)eth->ether_shost));
    printf(" ether destination : %s\n", ether_ntoa((struct ether_addr *)eth->ether_dhost));

    if (ntohs(eth->ether_type) == 0x0800) { // 0x0800 is IP type

        struct ipheader * ip = (struct ipheader *)
            (packet + sizeof(struct ethheader));

        printf(" ip source : %s\n", inet_ntoa(ip->iph_sourceip));
        printf(" ip destination : %s\n", inet_ntoa(ip->iph_destip));

        if(ip->iph_protocol == IPPROTO_TCP){
            int ip_header_len = ip->iph_ihl * 4;
            struct tcpheader *tcp = (struct tcpheader *)
                (packet + sizeof(struct ethheader) + ip_header_len);

            printf(" TCP source port : %d\n", ntohs(tcp->tcp_sport));
            printf(" TCP destination port : %d\n", ntohs(tcp->tcp_dport));

            int tcp_header_len = TH_OFF(tcp) * 4;
            int total_header_len = sizeof(struct ethheader) + ip_header_len + tcp_header_len;
        }
    }
}

```

```

printf(" =====\n");
printf(" HTTP Message \n");
printf("%.s\n", header->len - total_header_len , packet + total_header_len);
printf(" =====\n");

} else{
printf(" TCP가 아닙니다 \n");
}

}
}

int main()
{
pcap_t *handle;
char errbuf[PCAP_ERRBUF_SIZE];
struct bpf_program fp;
char filter_exp[] = "tcp port 80";
bpf_u_int32 net;

// Step 1: Open live pcap session on NIC with name enp0s3
handle = pcap_open_live("enp0s3", BUFSIZ, 1, 1000, errbuf);

// Step 2: Compile filter_exp into BPF psuedo-code
pcap_compile(handle, &fp, filter_exp, 0, net);
if (pcap_setfilter(handle, &fp) !=0) {
pcap_perror(handle, "Error:");
exit(EXIT_FAILURE);
}

// Step 3: Capture packets
pcap_loop(handle, -1, got_packet, NULL);

pcap_close(handle); //Close the handle

```

```
return 0;
}
```

→ sniff_improved.c와 myheader.h 를 참고해서 구조를 가져오고, 필요한 부분들을 추가 및 수정하였습니다.

[코드 추가 및 수정 사유]

- 헤더 파일

```
#include <stdlib.h>
#include <stdio.h>
#include <pcap.h>
#include <netinet/ether.h>
#include <arpa/inet.h>
```

1. #include <netinet/ether.h>

→ ether_ntoa 함수와 ether_addr 구조체를 사용하기 위해서 추가하였습니다.

- 구조체

```
/* Ethernet header */
struct ethheader {
    u_char ether_dhost[6]; /* destination host address */
    u_char ether_shost[6]; /* source host address */
    u_short ether_type;    /* IP? ARP? RARP? etc */
};

/* IP Header */
struct ipheader {
    unsigned char    iph_ihl:4, //IP header length
                    iph_ver:4; //IP version
```

```

unsigned char    iph_tos; //Type of service
unsigned short int iph_len; //IP Packet length (data + header)
unsigned short int iph_ident; //Identification
unsigned short int iph_flag:3, //Fragmentation flags
                iph_offset:13; //Flags offset
unsigned char    iph_ttl; //Time to Live
unsigned char    iph_protocol; //Protocol type
unsigned short int iph_chksum; //IP datagram checksum
struct in_addr   iph_sourceip; //Source IP address
struct in_addr   iph_destip; //Destination IP address
};

/* ICMP Header */
struct icmpheader {
    unsigned char icmp_type; // ICMP message type
    unsigned char icmp_code; // Error code
    unsigned short int icmp_chksum; //Checksum for ICMP Header and data
    unsigned short int icmp_id; //Used for identifying request
    unsigned short int icmp_seq; //Sequence number
};

/* TCP Header */
struct tcpheader {
    u_short tcp_sport;    /* source port */
    u_short tcp_dport;    /* destination port */
    u_int   tcp_seq;      /* sequence number */
    u_int   tcp_ack;      /* acknowledgement number */
    u_char  tcp_offx2;    /* data offset, rsvd */
#define TH_OFF(th)      (((th)→tcp_offx2 & 0xf0) >> 4)
    u_char  tcp_flags;
#define TH_FIN  0x01
#define TH_SYN  0x02
#define TH_RST  0x04
#define TH_PUSH 0x08
#define TH_ACK  0x10
#define TH_URG  0x20
#define TH_ECE  0x40
#define TH_CWR  0x80

```

```

#define TH_FLAGS      (TH_FIN|TH_SYN|TH_RST|TH_ACK|TH_URG|TH_ECE|
    u_short tcp_win;      /* window */
    u_short tcp_sum;      /* checksum */
    u_short tcp_urp;      /* urgent pointer */
};

/* Psuedo TCP header */
struct pseudo_tcp
{
    unsigned saddr, daddr;
    unsigned char mbz;
    unsigned char ptcl;
    unsigned short tcpl;
    struct tcpheader tcp;
    char payload[1500];
};

```

1. 필요 없는 구조체 삭제

→ TCP만 볼 것이기 때문에, ICMP와 UDP 구조체를 삭제하였습니다.

- got_packet 함수

```

void got_packet(u_char *args, const struct pcap_pkthdr *header,
               const u_char *packet)
{
    struct ethheader *eth = (struct ethheader *)packet;

    printf(" ether source : %s\n", ether_ntoa((struct ether_addr *)eth->ether_shost));
    printf(" ether destination : %s\n", ether_ntoa((struct ether_addr *)eth->ether_dhost));

    if (ntohs(eth->ether_type) == 0x0800) { // 0x0800 is IP type

        struct ipheader *ip = (struct ipheader *)

```

```

(packet + sizeof(struct ethheader));

printf(" ip source : %s\n", inet_ntoa(ip→iph_sourceip));
printf(" ip destination : %s\n", inet_ntoa(ip→iph_destip));

if(ip→iph_protocol == IPPROTO_TCP){
int ip_header_len = ip→iph_ihl * 4;
struct tcpheader *tcp = (struct tcpheader *)
(packet + sizeof(struct ethheader) + ip_header_len);

printf(" TCP source port : %d\n", ntohs(tcp→tcp_sport));
printf(" TCP destination port : %d\n", ntohs(tcp→tcp_dport));

int tcp_header_len = TH_OFF(tcp) * 4;
int total_header_len = sizeof(struct ethheader) + ip_header_len + tcp_header_len;

printf(" =====\n");
printf(" HTTP Message \n");
printf("%.s\n", header→len - total_header_len , packet + total_header_len);
printf(" =====\n");

} else{
printf(" TCP가 아닙니다 \n");
}
}
}

```

1. **printf(" ether source : %s\n", ether_ntoa((struct ether_addr *)eth->ether_shost));**

→ src mac을 출력하기 위해 작성하였습니다

→ ether_ntoa 함수로 MAC 주소를 사람이 읽을 수 있는 문자열 형태로 변환해주었습니다

2. **printf(" ether destination : %s\n", ether_ntoa((struct ether_addr *)eth->ether_dhost));**

→ dst mac을 출력하기 위해 작성하였습니다.

→ ether_ntoa 함수로 MAC 주소를 사람이 읽을 수 있는 문자열 형태로 변환해주었습니다

3. Switch 문

~~→ 결과에서는 필요가 없으나, 제가 TCP로 잘 접근했는지 확인하기 위해 유지하였습니다.~~

~~→ return을 break로 바꿔주어, 다음 코드로 넘어가도록 수정하였습니다.~~

→ 진짜 필요가 없어서, 보고서를 작성하는 과정에서 삭제하였습니다.

4. if(ip→iph_protocol == IPPROTO_TCP), else{" TCP가 아닙니다 \n")

→ TCP 프로토콜로 접속하면 그 다음 과정을 수행하도록 if를, TCP로 접속하지 못했다면 그 사실을 알려주기 위해 else를 사용하였습니다.

5. int ip_header_len = ip→iph_ihl * 4;

→ ip header의 길이는 기본적으로 4바이트 단위 블록 5개로 시작하는데, iph_ihl이 블록 개수가 저장되는 변수이므로, *4를 해줌으로써 가변적인 IP 헤더의 길이를 계산할 수 있도록 했습니다.

6. struct tcpheader *tcp = (struct tcpheader *) (packet + sizeof(struct ethheader) + ip_header_len);

→ packet은 바이트 단위 포인터이기 때문에, 정수를 더하면 해당 바이트 만큼 뒤로 이동하게 됩니다.

→ etherheader 사이즈와 ip_header_len 사이즈만큼 더해서 TCP 헤더의 시작 지점으로 이동하였습니다.

7. int tcp_header_len = TH_OFF(tcp) * 4;

→ TCP 헤더의 길이도 가변적입니다.

→ TH_OFF 함수를 통해서 4바이트 블록 몇 개를 사용하는가를, 비트 연산을 통해 알아내고, 이 블록 역시 4바이트이기 때문에 * 4를 해주어서 가변적인 TCP 헤더 길이를 계산하였습니다.

8. **int total_header_len = sizeof(struct ethheader) + ip_header_len + tcp_header_len;**

→ 패킷에 붙은 모든 헤더 길이를 담은 변수를 선언해주었습니다.

9. **printf("=====\n");**
printf(" HTTP Message \n");
printf("=====\n");

→ 출력했을 때의 가독성을 위해서 작성해주었습니다.

10. **printf("%.s\n", header->len - total_header_len , packet + total_header_len);**

→ 메시지를 출력하기 위해 작성해주었습니다.

→ printf는 널 문자까지 출력을 해주는데, 메시지는 널 문자로 끝나지 않는다고 알고 있습니다. 그래서 제가 지정한 만큼 출력해주기 위해서 %.s 형식 지정자를 이용하였습니다.

→ pcap_pkthdr 구조를 가진 header에 len 이라는 패킷 전체 길이를 가진 변수가 있어서, 'header->len - total_header_len'을 통해, 딱 메시지 길이만큼 수를 설정할 수 있었습니다.

→ packet + total_header_len으로, 메시지 시작점부터 출력하도록 설정해주었습니다.

- main 함수

```
int main()
{
    pcap_t *handle;
    char errbuf[PCAP_ERRBUF_SIZE];
    struct bpf_program fp;
    char filter_exp[] = "tcp port 80";
    bpf_u_int32 net;

    // Step 1: Open live pcap session on NIC with name enp0s3
```

```

handle = pcap_open_live("enp0s3", BUFSIZ, 1, 1000, errbuf);

// Step 2: Compile filter_exp into BPF psuedo-code
pcap_compile(handle, &fp, filter_exp, 0, net);
if (pcap_setfilter(handle, &fp) !=0) {
    pcap_perror(handle, "Error:");
    exit(EXIT_FAILURE);
}

// Step 3: Capture packets
pcap_loop(handle, -1, got_packet, NULL);

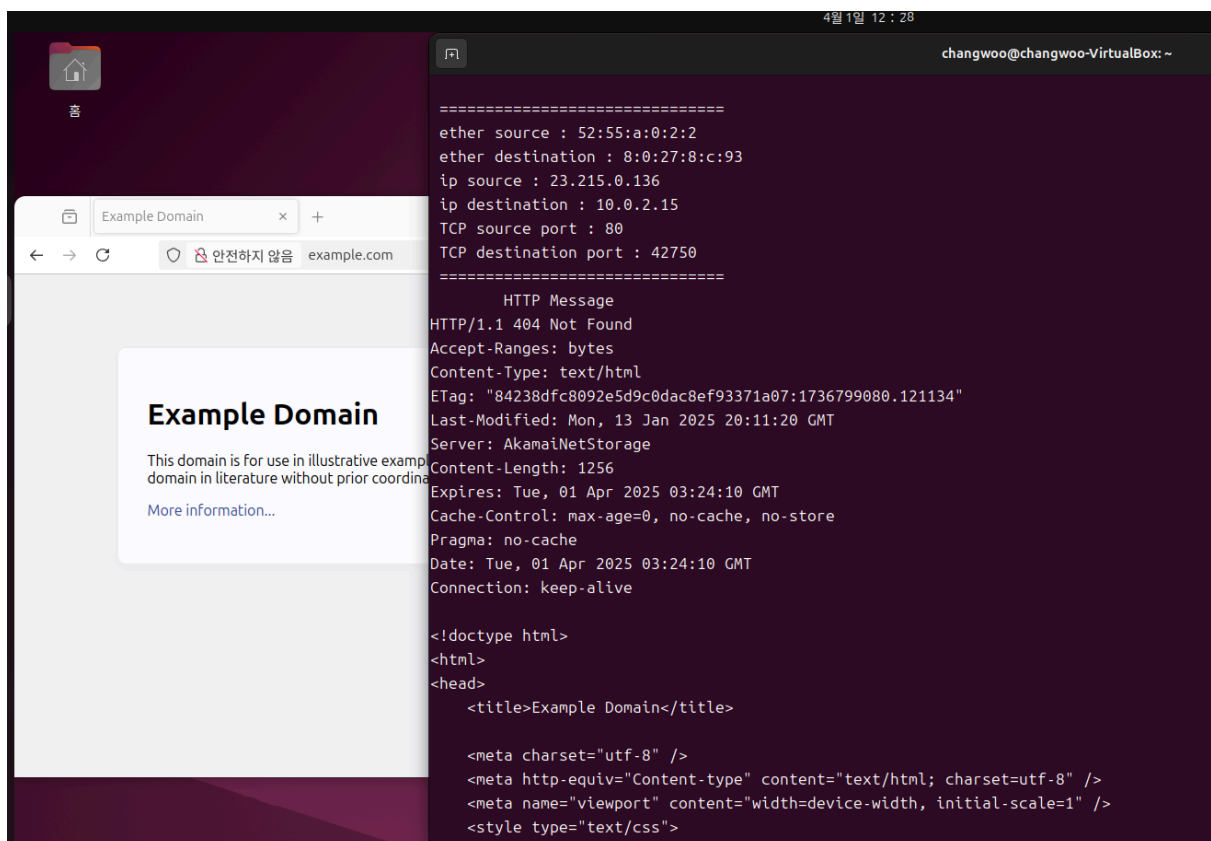
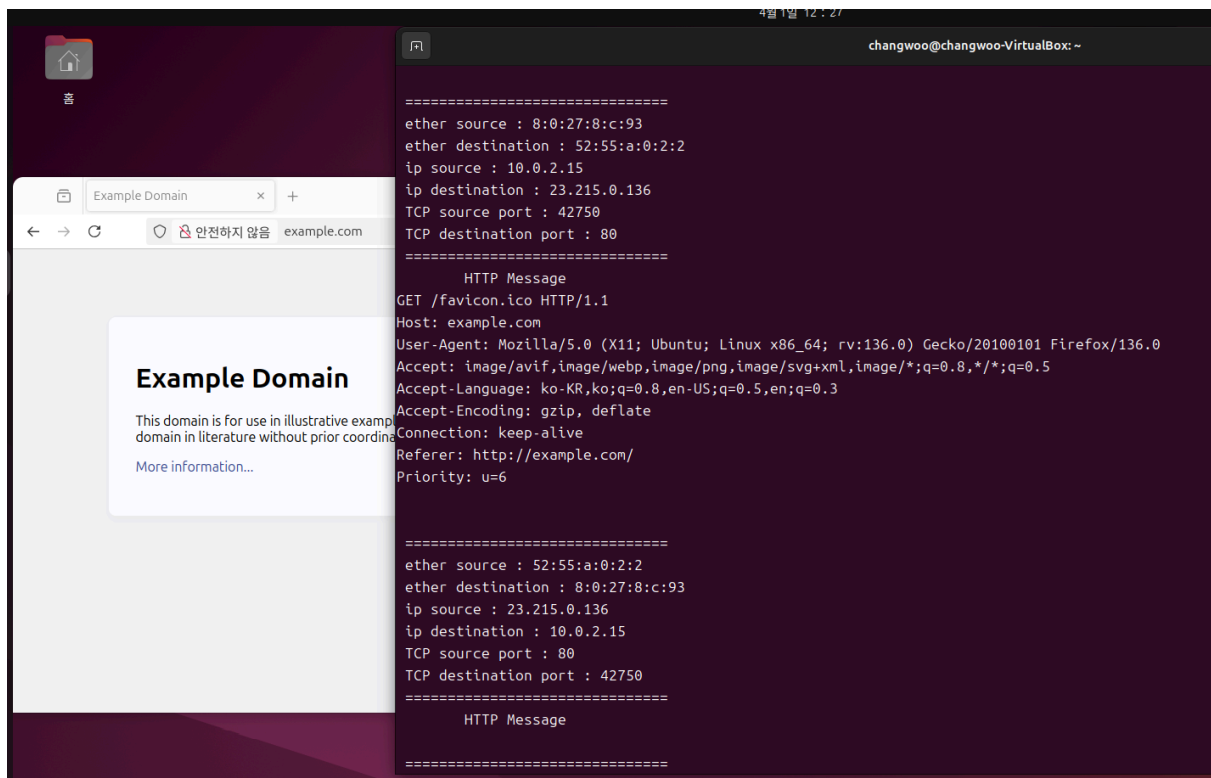
pcap_close(handle); //Close the handle
return 0;
}

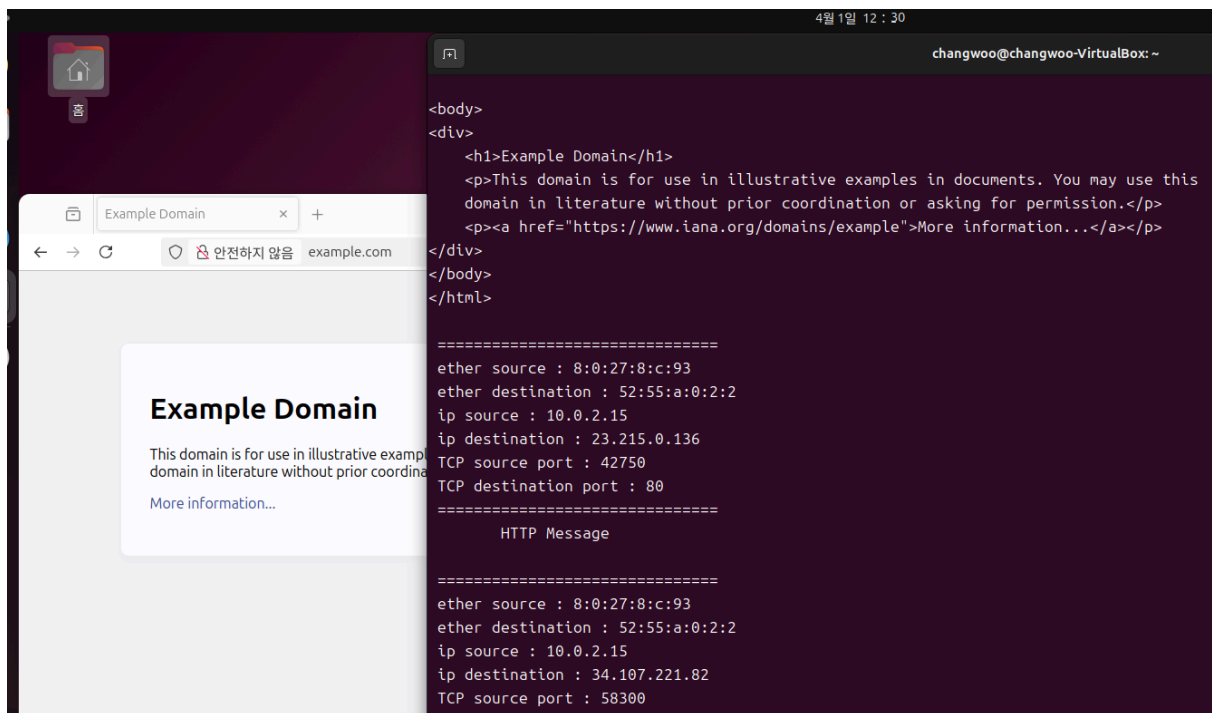
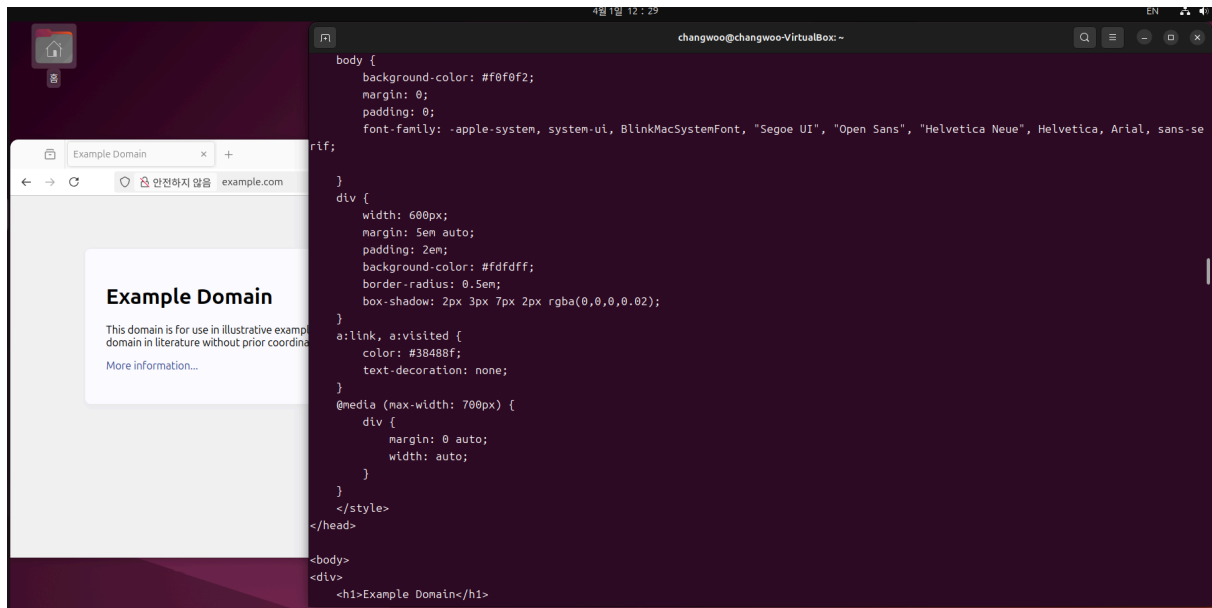
```

1. **char filter_exp[] = "tcp port 80";**

→ 저는 TCP 프로토콜로 http만 확인할 것이기 때문에, TCP프로토콜 중에서 80번 포트만 확인해주기 위해 수정하였습니다.

[결과]





→ 제가 원하는 방향으로 모든 메시지가 출력 되었습니다!

[과제 후기]

누군가에게 정말 아무것도 아니었겠지만, 네트워크 지식과 프로그래밍 지식이 전무한 수준인 제겐 큰 도전이었습니다. 모르는 개념과 헷갈리는 개념들을 열심히 서칭해보고, 때로는

GPT에게 물어보기도 하면서, 생활에 필요한 시간을 제외한 이틀 조금 넘게 걸쳐 프로그래밍을 완료하였습니다!

아직 부족한 부분들이 엄청엄청 많지만 과제를 해가면서 당연히 네트워크에 대한 기초적인 지식들도 습득할 수 있었고, 프로그래밍에 대한 지식도 조금이나마 같이 얻어서 정말 유익한 시간이었다고 느꼈습니다. 감사합니다!